

Tarea 07 de Lenguajes de Programación

Ramírez Ríos, Víctor Hugo
a221205528@unison.mx

Noviembre 2023

1. Extiende el lenguaje agregando un nuevo operador minus que toma un argumento n y regresa $-n$. Por ejemplo, el valor de `minus(-(minus(5),9))` debe ser 14.

- Léxico
minus
- Sintaxis
 - Concreta
 $Expression \rightarrow \text{minus}(Expression)$
 - Abstracta
`(minus-exp exp1)`
- Semántica
 $(\text{value-of}(\text{minus-exp } exp1)\rho) = (\text{num-val } (- (\text{expval} \rightarrow \text{num } (\text{value-of } exp1 \rho)))$

2. Extiende el lenguaje agregando operadores para la suma, multiplicación y cociente de enteros.

- Léxico
+
*
/
■ Sintaxis
 - Concreta
 $Expression \rightarrow +(Expression, Expression)$
 $Expression \rightarrow *(Expression, Expression)$
 $Expression \rightarrow /(Expression, Expression)$
 - Abstracta
`(sum-exp exp1 exp2)`
`(mult-exp exp1 exp2)`
`(div-exp exp1 exp2)`
- Semántica

$$\begin{aligned} & (\text{value-of}(\text{sum-exp } exp1 exp2)\rho) \\ &= (\text{num-val}(+(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of } exp2 \rho)))) \end{aligned}$$

$$\begin{aligned} & (\text{value-of}(\text{mult-exp } exp1 exp2)\rho) \\ &= (\text{num-val}(*(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of } exp2 \rho)))) \end{aligned}$$

$$\begin{aligned} & (\text{value-of}(\text{div-exp } exp1 exp2)\rho) \\ &= (\text{num-val}(\text{floor}/(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of } exp2 \rho))))) \end{aligned}$$

3. Agrega un predicado de igualdad numérica `equal?` y predicados de orden `greater?` y `less?` al conjunto de operaciones del lenguaje LET.

- Léxico
 - `equal?`
 - `greater?`
 - `less?`
- Sintaxis
 - Concreta
 - $Expression \rightarrow \text{equal?}(Expression, Expression)$
 - $Expression \rightarrow \text{greater?}(Expression, Expression)$
 - $Expression \rightarrow \text{less?}(Expression, Expression)$
 - Abstracta
 - $(\text{equal?}-\text{exp } exp1 \ exp2)$
 - $(\text{greater?}-\text{exp } exp1 \ exp2)$
 - $(\text{less?}-\text{exp } exp1 \ exp2)$
- Semántica

$$\begin{aligned} & (\text{value-of}(\text{equal?}-\text{exp } exp1 \ exp2) \rho) \\ &= (\text{num-val}(\text{equal?}(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of } exp2 \rho)))) \end{aligned}$$

$$\begin{aligned} & (\text{value-of}(\text{greater?}-\text{exp } exp1 \ exp2) \rho) \\ &= (\text{num-val}(> (\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of } exp2 \rho)))) \end{aligned}$$

$$\begin{aligned} & (\text{value-of}(\text{less?}-\text{exp } exp1 \ exp2) \rho) \\ &= (\text{num-val}(< (\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)) \\ & \quad (\text{expval} \rightarrow \text{num}(\text{value-of } exp2 \rho)))) \end{aligned}$$

4. Agrega operaciones de procesamiento de listas al lenguaje, incluyendo `cons`, `car`, `cdr`, `null?` y `emptylist`.

- Léxico
 - `cons`
 - `car`
 - `cdr`
 - `null?`
 - `emptylist`
- Sintaxis
 - Concreta
 - $Expression \rightarrow \text{cons}(Expression, Expression)$
 - $Expression \rightarrow \text{car}(Expression)$
 - $Expression \rightarrow \text{cdr}(Expression)$
 - $Expression \rightarrow \text{null?}(Expression)$
 - $Expression \rightarrow \text{emptylist}()$
 - Abstracta
 - $(\text{cons}-\text{exp } exp1 \ exp2)$
 - $(\text{car}-\text{exp } exp1)$
 - $(\text{cdr}-\text{exp } exp1)$
 - $(\text{null?}-\text{exp } exp1)$
 - $(\text{emptylist}-\text{exp } exp1)$

- Semántica

$$\begin{aligned}
& (\text{value-of}(\text{cons-exp } exp1 exp2) \rho) \\
&= (\text{let}([val1(\text{value-of } exp1 \rho)] \\
&\quad [val2(\text{value-of } exp2 \rho)]) \\
&\quad (\text{pair-val}(\text{cons } val1 val2)))
\end{aligned}$$

$$\begin{aligned}
& (\text{value-of}(\text{car-exp } exp1) \rho) \\
&= (\text{pair-val}(\text{car}(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)))
\end{aligned}$$

$$\begin{aligned}
& (\text{value-of}(\text{cdr-exp } exp1) \rho) \\
&= (\text{pair-val}(\text{cdr}(\text{expval} \rightarrow \text{num}(\text{value-of } exp1 \rho)))
\end{aligned}$$

$$\begin{aligned}
& (\text{value-of}(\text{null?-exp } exp1) \rho) \\
&= (\text{let}([val1(\text{value-of } exp1 \rho)]) \\
&\quad (\text{bool-val}(= \text{null-val } val1)))
\end{aligned}$$

$$\begin{aligned}
& (\text{value-of}(\text{emptylist-exp } exp1) \rho) \\
&= (\text{null-val})
\end{aligned}$$

5. Agrega una operacion list al lenguaje. Esta operacion debe tomar cualquier cantidad de argumentos y regresar un valor expresado de la lista de sus valores.

- Léxico

list

- Sintaxis

- Concreta

$Expression \rightarrow \text{list}(Expressions)$

$Expressions \rightarrow \epsilon$

$Expressions \rightarrow ExpressionsAux$

$ExpressionsAux \rightarrow Expression$

$ExpressionsAux \rightarrow Expression, ExpressionsAux$

- Abstracta

$(\text{list-exp } exps)$

- Semántica

$$\begin{aligned}
& (\text{value-of}(\text{list-exp } exps) \rho) \\
&= (\text{let}([val1(\text{value-of } exps \rho)]) \\
&\quad (\text{if}(\text{expval} \rightarrow \text{bool}(= \text{null-val } val1)) \\
&\quad (\text{null-val}) \\
&\quad (\text{haz una lista con el cons, car y cdr})))
\end{aligned}$$

6. Incorpora al lenguaje expresiones cond. Usa la gramatica

$$Expression \rightarrow \mathbf{cond}\{Expression \Rightarrow Expression\} * \mathbf{end}$$

- Léxico
 - cond
 - end
 - $\Rightarrow$
- Sintaxis
 - Concreta

$$Expression \rightarrow \mathbf{cond}\{Expression \Rightarrow Expression\} * \mathbf{end}$$

- Abstracta
(cond-exp *exprs*)
- Semántica

7. Cambia los valores del lenguaje para que los enteros sean los unicos valores expresados. Modifica if para que el valor de 0 sea tratado como falso y todos los otros valores sean tratados como verdaderos. Modifica los predicados de manera consistente.

$$\varepsilon(\mathbf{zero?}(exp1), \rho) = \begin{cases} 1 & \text{si } \varepsilon(exp1, \rho) = 0 \\ 0 & \text{si } \varepsilon(exp1, \rho) \neq 0 \end{cases}$$

$$\varepsilon(\mathbf{if}(exp1)\mathbf{then}(exp2)\mathbf{else}(exp3), \rho) = \begin{cases} \varepsilon(exp2, \rho) & \text{si } \varepsilon(exp1, \rho) = 1 \\ \varepsilon(exp3, \rho) & \text{si } \varepsilon(exp1, \rho) = 0 \end{cases}$$

8. Como una alternativa al ejercicio anterior, agrega una nueva categoria sintactica Bool-exp de expresiones booleanas al lenguaje. Cambia la producción para expresiones condicionales para que sea

$$Expression \rightarrow \mathbf{if} \text{ Bool-exp } \mathbf{then} expression \mathbf{else} Expression$$

Escribe producciones apropiadas para Bool-exp y especifica su semantica con value-of-bool-exp. En donde terminan estando los predicados del ejercicio 3 con este cambio?

- Léxico
- Sintaxis
 - Concreta

$$Expression \rightarrow \mathbf{if} \text{ Bool-exp } \mathbf{then} expression \mathbf{else} Expression$$

$$\text{Bool-exp} = 1$$

$$\text{Bool-exp} = 0$$

$$\text{Bool-exp} = \mathbf{zero?}(Expression)$$

$$\text{Bool-exp} = \mathbf{equal?}(Expression, Expression)$$

$$\text{Bool-exp} = \mathbf{greater?}(Expression, Expression)$$

$$\text{Bool-exp} = 0$$

- Abstracta
 (if boolexp exp1 exp2)
 (bool-exp *bool*)
 (zero?-boolexp *exp1*)
 (equal?-boolexp *exp1 exp2*)
 (greater?-boolexp *exp1 exp2*)
 (less?-boolexp *exp1 exp2*)

■ Semántica

9. Modifica la implementacion del interprete agregando una nueva operacion print que toma un argumento, lo imprime y regresa el entero 1. Por que esta expresion no es expresable en nuestro metodo de especificacion formal?

Por que no tenemos una forma de representar la operacion de impresión usando la especificación formal.

10. Extiende el lenguaje para que las expresiones let puedan vincular una cantidad arbitraria de variables, usando la producción,

$$Expression \rightarrow \mathbf{let}\{Identifier = Expression\} * \mathbf{in}Expression$$

11. Extiende el lenguaje con una expresión let* que funciona como en Racket.
12. Agrega una expresión al lenguaje de acuerdo a la siguiente regla,

$$Expression \rightarrow \mathbf{unpack}\{Identifier\} * = Expression \mathbf{in}Expression$$

13. Consideremos funciones de compilación compile-program y compile que toman de argumento un árbol de sintaxis y regresan un árbol de sintaxis. Implementa estas funciones para que el lenguaje compile el cond del ejercicio 7, la extensión a let del ejercicio 11, y el let* del ejercicio 12, utilizando el lenguaje LET base y las extensiones del lenguaje 4,5 y 13.